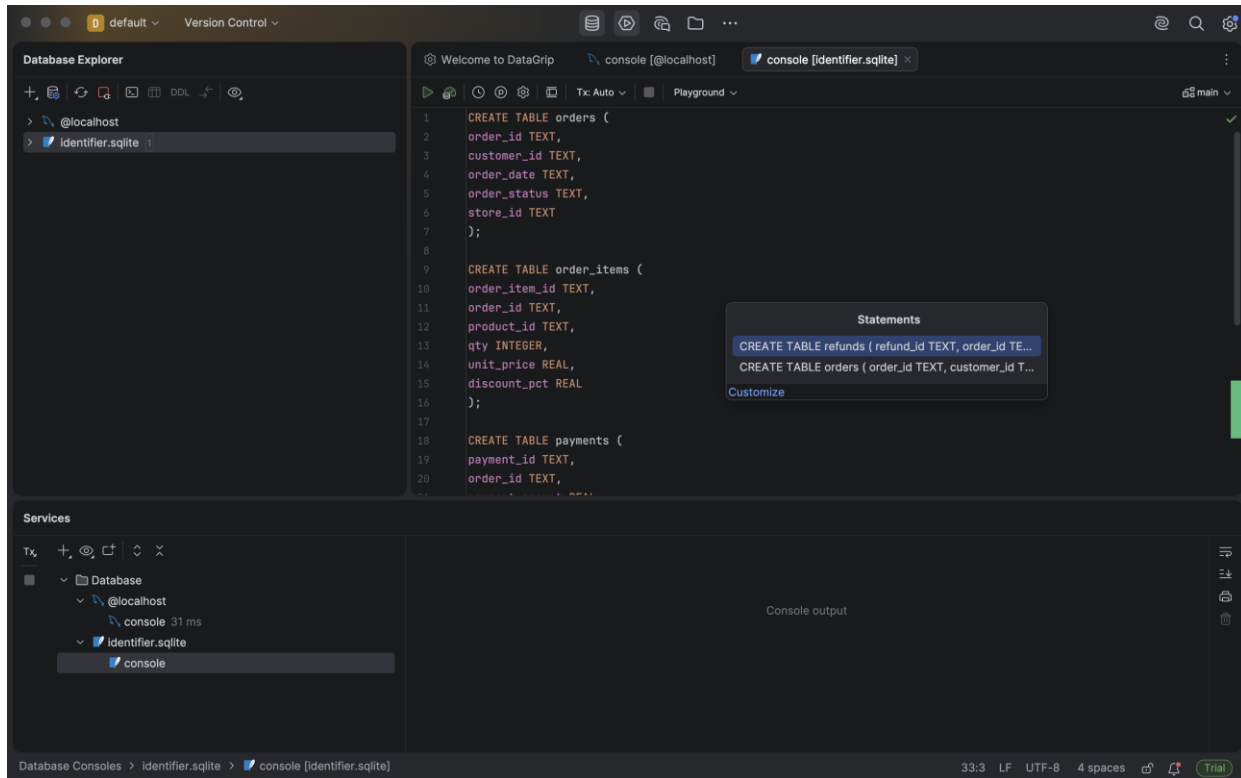


ITHENA TECHNICAL EVALUATION ASSIGNMENT

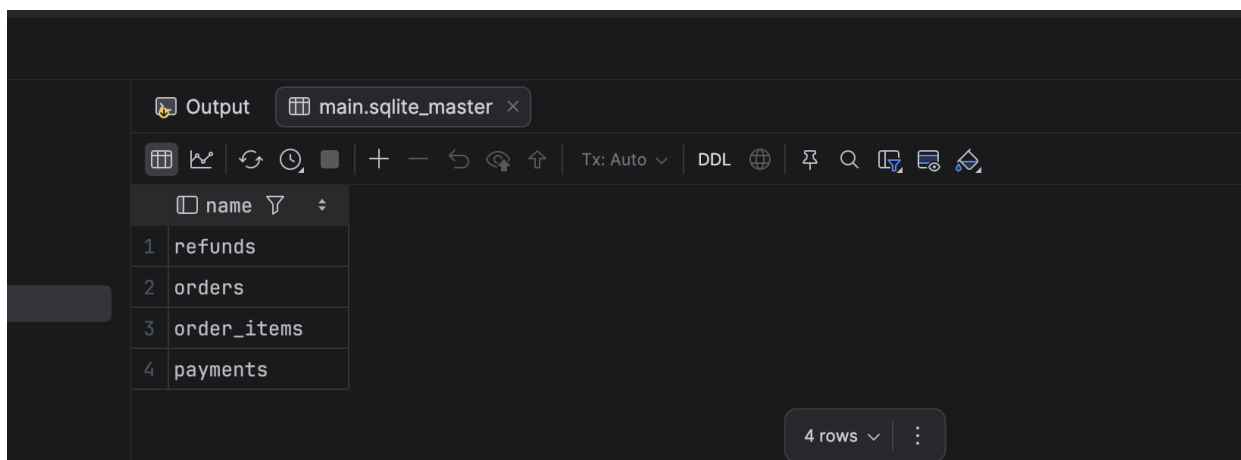
Project 1- SQL/Python Results

QUESTION 4:

1. Created Table from Given Schema:



2. Confirming all tables exists:



3. Inserting Sample Data into Tables

The screenshot displays the DataGrip interface with a SQL editor containing 22 lines of INSERT statements. The statements are organized into four groups: orders (lines 1-6), order_items (lines 8-13), payments (lines 15-20), and refunds (line 22). Each statement includes column names and values in single quotes. The interface also shows a Database Explorer on the left, a Services panel at the bottom left, and an Output panel at the bottom right showing a table with 4 rows.

```
1 INSERT INTO orders VALUES ( order_id 'ORD1', customer_id 'CUST1', order_date '2024-01-05', order_status 'completed',
2 INSERT INTO orders VALUES ( order_id 'ORD2', customer_id 'CUST2', order_date '2024-01-06', order_status 'completed',
3 INSERT INTO orders VALUES ( order_id 'ORD3', customer_id 'CUST3', order_date '2024-01-07', order_status 'completed',
4 INSERT INTO orders VALUES ( order_id 'ORD4', customer_id 'CUST4', order_date '2024-01-08', order_status 'completed',
5 INSERT INTO orders VALUES ( order_id 'ORD5', customer_id 'CUST5', order_date '2024-01-09', order_status 'completed',
6 INSERT INTO orders VALUES ( order_id 'ORD6', customer_id 'CUST6', order_date '2024-01-10', order_status 'completed',
7
8 INSERT INTO order_items VALUES ( order_item_id 'ITEM1', order_id 'ORD1', product_id 'PROD1', qty 2, unit_price 50.00, di
9 INSERT INTO order_items VALUES ( order_item_id 'ITEM2', order_id 'ORD2', product_id 'PROD2', qty 1, unit_price 100.00, t
10 INSERT INTO order_items VALUES ( order_item_id 'ITEM3', order_id 'ORD3', product_id 'PROD3', qty 3, unit_price 30.00, di
11 INSERT INTO order_items VALUES ( order_item_id 'ITEM4', order_id 'ORD4', product_id 'PROD4', qty 1, unit_price 200.00, t
12 INSERT INTO order_items VALUES ( order_item_id 'ITEM5', order_id 'ORD5', product_id 'PROD5', qty 5, unit_price 20.00, di
13 INSERT INTO order_items VALUES ( order_item_id 'ITEM6', order_id 'ORD6', product_id 'PROD6', qty 2, unit_price 75.00, di
14
15 INSERT INTO payments VALUES ( payment_id 'PAY1', order_id 'ORD1', payment_amount 100.00, payment_method 'card', pay
16 INSERT INTO payments VALUES ( payment_id 'PAY2', order_id 'ORD2', payment_amount 90.00, payment_method 'card', paym
17 INSERT INTO payments VALUES ( payment_id 'PAY3', order_id 'ORD3', payment_amount 90.00, payment_method 'card', paym
18 INSERT INTO payments VALUES ( payment_id 'PAY3B', order_id 'ORD3', payment_amount 90.00, payment_method 'card', pay
19 INSERT INTO payments VALUES ( payment_id 'PAY4', order_id 'ORD4', payment_amount 200.00, payment_method 'card', pay
20 INSERT INTO payments VALUES ( payment_id 'PAY5', order_id 'ORD5', payment_amount 120.00, payment_method 'card', pay
21
22 INSERT INTO refunds VALUES ( refund_id 'REF1', order_id 'ORD4', refund_amount 20.00, refund_date '2024-01-09', refund_id
```

order_id	order_value	total_paid	total_refunded
ORD1	100.00	100.00	0.00
ORD2	90.00	90.00	0.00
ORD3	90.00	90.00	0.00
ORD4	200.00	200.00	20.00
ORD5	120.00	120.00	0.00

4. Confirming inputs of the tables:

The screenshot shows the DataGrip IDE interface. The top-left pane is the 'Database Explorer' showing a connection to '@localhost' and a database named 'identifier.sqlite'. The top-right pane shows the SQL editor with a query that counts rows in four tables: 'orders', 'order_items', 'payments', and 'refunds'. The bottom-left pane shows the 'Services' section with a tree view of the database structure. The bottom-right pane shows the 'Output' section with a table of results.

```
1 SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM orders
2 UNION ALL
3 SELECT table_name 'order_items', row_count COUNT(*) FROM order_items
4 UNION ALL
5 SELECT table_name 'payments', row_count COUNT(*) FROM payments
6 UNION ALL
7 SELECT table_name 'refunds', row_count COUNT(*) FROM refunds;
```

table_name	row_count
orders	6
order_items	6
payments	6
refunds	1

5. Running the reconciliation theory/Results:
(The entire query is pasted in Technical-Questions.sql under project-1-sql-python)

The screenshot shows the DataGrip IDE interface. The main editor displays a SQL query for reconciliation. The query joins order items with payments and refunds to calculate the difference between total paid and total refunded amounts for each order.

```
9 FROM order_items
10 GROUP BY order_id
11 ) ov ON ov.order_id = o.order_id
12 JOIN (
13 SELECT order_id, SUM(payment_amount) AS total_paid
14 FROM payments
15 WHERE payment_status = 'success'
16 GROUP BY order_id
17 ) p ON p.order_id = o.order_id
18 LEFT JOIN (
19 SELECT order_id, SUM(refund_amount) AS total_refunded
20 FROM refunds
21 GROUP BY order_id
22 ) r ON r.order_id = o.order_id
23 WHERE ABS(ov.order_value - (p.total_paid - COALESCE(r.total_refunded, 0))) > 1;
```

A tooltip shows the definition of the alias `ov.order_value`:

```
SUM(qty * unit_price) AS order_value
```

The bottom panel shows the query results in a table with 5 rows and 5 columns:

	order_id	order_value	total_paid	total_refunded
1	ORD2		100	90
2	ORD3		90	180
3	ORD4		200	200
4	ORD5		100	120

The status bar at the bottom indicates the file is `console [identifier.sqlite]` with encoding `UTF-8` and 4 spaces.

QUESTION 5:

1. Running the de-duplication theory/Results:

The screenshot shows the DataGrip IDE interface. The top panel displays a SQL query in the console:

```
1 SELECT p1.payment_id, p1.order_id, p1.payment_amount, p1.payment_date 356 ms
2 FROM payments p1
3 JOIN payments p2
4 ON p1.order_id = p2.order_id
5 AND p1.payment_amount = p2.payment_amount
6 AND p1.payment_id != p2.payment_id
7 AND ABS(strftime('%s', p1.payment_date) - strftime('%s', p2.payment_date)) <= 60
8 AND p1.payment_id > p2.payment_id;
```

The bottom panel shows the output of the query, titled "main.payments". The output is a table with the following columns: payment_id, order_id, payment_amount, and payment_date. The first row of data is:

payment_id	order_id	payment_amount	payment_date
PAY3B	ORD3	90	2024-01-07 09:00:20

The status bar at the bottom indicates the file is "identifier.sqlite" and the console is "console [identifier.sqlite]". The bottom right corner shows the time "8:35", encoding "LF UTF-8", and "4 spaces".

QUESTION 6:

```
Last login: Sun Aug 2 19:31:10 on ttys000
[sudo@Sayyams-MacBook-Pro ~ % python3
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import pandas as pd
... orders_df = pd.DataFrame([
...     ("ORD1", "CUST1", "2024-01-05", "completed", "STORE1"),
...     ("ORD2", "CUST2", "2024-01-06", "completed", "STORE1"),
...     ("ORD3", "CUST3", "2024-01-07", "completed", "STORE2"),
...     ("ORD4", "CUST4", "2024-01-08", "completed", "STORE2"),
...     ("ORD5", "CUST5", "2024-01-09", "completed", "STORE1"),
...     ("ORD6", "CUST6", "2024-01-10", "completed", "STORE2"),
... ], columns=["order_id", "customer_id", "order_date", "order_status", "store_id"])
... items_df = pd.DataFrame([
...     ("ITEM1", "ORD1", "PROD1", 2, 50.00, 0),
...     ("ITEM2", "ORD2", "PROD2", 1, 100.00, 0),
...     ("ITEM3", "ORD3", "PROD3", 3, 30.00, 0),
...     ("ITEM4", "ORD4", "PROD4", 1, 200.00, 0),
...     ("ITEM5", "ORD5", "PROD5", 5, 20.00, 0),
...     ("ITEM6", "ORD6", "PROD6", 2, 75.00, 0),
... ], columns=["order_item_id", "order_id", "product_id", "qty", "unit_price", "discount_pct"])
... payments_df = pd.DataFrame([
...     ("PAY1", "ORD1", 100.00, "card", "2024-01-05 10:00:00", "success"),
...     ("PAY2", "ORD2", 90.00, "card", "2024-01-06 11:00:00", "success"),
...     ("PAY3", "ORD3", 90.00, "card", "2024-01-07 09:00:00", "success"),
...     ("PAY3B", "ORD3", 90.00, "card", "2024-01-07 09:00:20", "success"),
...     ("PAY4", "ORD4", 200.00, "card", "2024-01-08 14:00:00", "success"),
...     ("PAY5", "ORD5", 120.00, "card", "2024-01-09 15:00:00", "success"),
... ], columns=["payment_id", "order_id", "payment_amount", "payment_method", "payment_date", "payment_status"])
... refunds_df = pd.DataFrame([
...     ("REF1", "ORD4", 20.00, "2024-01-09", "customer changed mind"),
... ], columns=["refund_id", "order_id", "refund_amount", "refund_date", "refund_reason"])
... items_df["line_total"] = items_df["qty"] * items_df["unit_price"] * (1 - items_df["discount_pct"])
... order_value = items_df.groupby("order_id")["line_total"].sum().reset_index(name="order_value")
... paid = payments_df[payments_df["payment_status"] == "success"]
... total_paid = paid.groupby("order_id")["payment_amount"].sum().reset_index(name="total_paid")
... total_refunded = refunds_df.groupby("order_id")["refund_amount"].sum().reset_index(name="total_refunded")
... df = orders_df[["order_id"]].merge(order_value, on="order_id", how="left").merge(total_paid, on="order_id", how="left").merge(total_refunded, on="order_id", how="left").fillna(0)
... df["net_received"] = df["total_paid"] - df["total_refunded"]
... df["difference"] = df["order_value"] - df["net_received"]
... tolerance = 1.0
... flagged = df[df["difference"].abs() > tolerance].copy()
... flagged["flag_reason"] = "overpayment"
... flagged.loc[flagged["difference"] > 0, "flag_reason"] = "underpayment"
... flagged.loc[flagged["total_refunded"] > 0, "flag_reason"] = "refund_mismatch"
... print(flagged.to_string(index=False))
[...
order_id  order_value  total_paid  total_refunded  net_received  difference  flag_reason
ORD2      100.0      90.0          0.0          90.0      10.0      underpayment
ORD3       90.0      180.0          0.0          180.0     -90.0      overpayment
ORD4      200.0      200.0         20.0          180.0      20.0  refund_mismatch
ORD5      100.0      120.0          0.0          120.0     -20.0      overpayment
ORD6      150.0       0.0          0.0           0.0     150.0      underpayment
>>> ]
```